

Spring boot allows easy integration with unit testing frameworks : JUnit 5(Jupiter) n Mockito.

Regarding Spring Boot Testing (Data Layer)

Spring Boot provides the `@DataJpaTest` annotation to test the persistence layer components.

It scans the `@Entity` classes and Spring Data JPA repositories.

Sets the `spring.jpa.show-sql` property to true and enable the SQL queries logging.

By Default, JPA test data are transactional and roll back at the end of each test; it means we do not need to clean up saved or modified table data after each test.

Replace the application data source, run and configure the embedded database (in memory eg : H2) on run time classpath(build path)

By default they will autoconfigure in-memory embedded database for testing purposes.
eg H2 database

But if you don't want to replace your original database by test database ,use
`@AutoConfigureTestDatabase(replace = Replace.NONE)`

Since test cases should not modify the state of DB , by default rollback is applied after the execution of test cases.

But , since we wanted to write test cases to insert the recs in emps table , marked
`@Rollback(false)`

`@DataJpaTest` annotation doesn't load other Spring beans (`@Components`, `@Controller`, `@Service`, and annotated beans) into `ApplicationContext(SC)`

By default, it scans for `@Entity` classes and configures Spring Data JPA repositories annotated with `@Repository` annotation.

2. Spring Boot provides
`@SpringBootTest` for integration testing
(eg : service +dao + DB)

Using `TestRestTemplate` - for controller + service +dao + DB
end - to -end testing